

Deep Learning

Assignment 8

Use MNIST Fashion Dataset and create a classifier to classify fashion clothing into categories. Using CNN.

Name: Rohit Kalaskar

Roll No: 24

PRN: 12310106

Branch: CS AI - B

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

from sklearn.metrics import classification_report, confusion_matrix

print(f'TensorFlow version: {tf.__version__}')
print(f'Keras version: {keras.__version__}')
```

```
TensorFlow version: 2.19.0
Keras version: 3.13.2
```

2. Load and Explore the Dataset

```
# Load Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()

# Class labels
class_names = [
    'T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
    'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot'
]

print(f'Training samples : {X_train.shape[0]}')
print(f'Test samples      : {X_test.shape[0]}')
print(f'Image shape       : {X_train.shape[1:]} (Height x Width)')
print(f'Number of classes: {len(class_names)}')
```

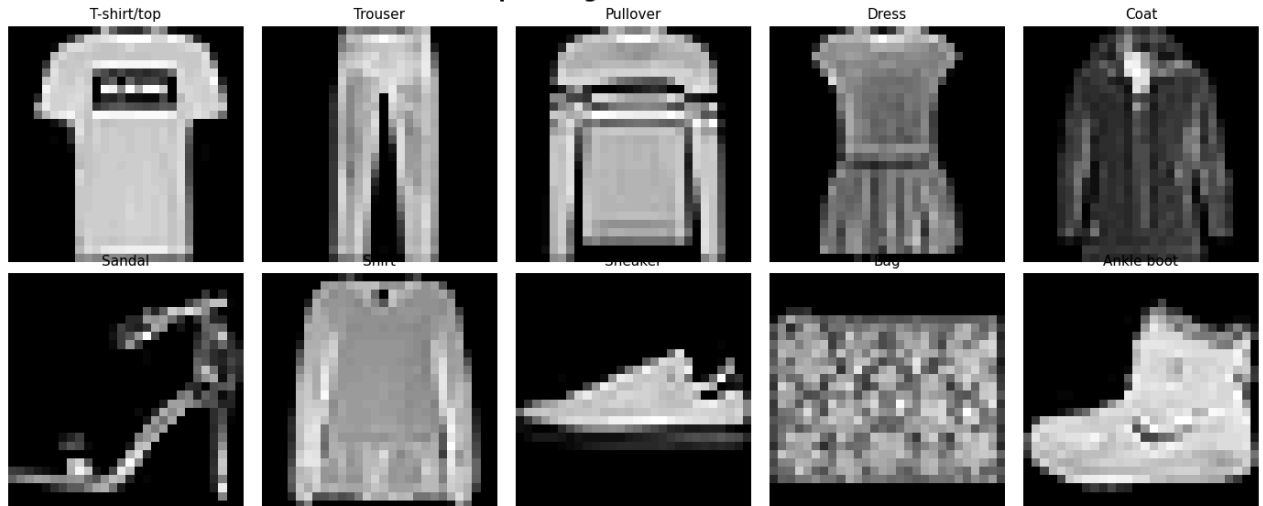
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 0s 0us/step
Training samples : 60000
Test samples      : 10000
Image shape       : (28, 28) (Height x Width)
Number of classes: 10
```

```
# Visualize sample images from each class
fig, axes = plt.subplots(2, 5, figsize=(14, 6))
fig.suptitle('Sample Images - One Per Class', fontsize=16, fontweight='bold')

for i, ax in enumerate(axes.flat):
    idx = np.where(y_train == i)[0][0]
    ax.imshow(X_train[idx], cmap='gray')
    ax.set_title(class_names[i], fontsize=11)
    ax.axis('off')

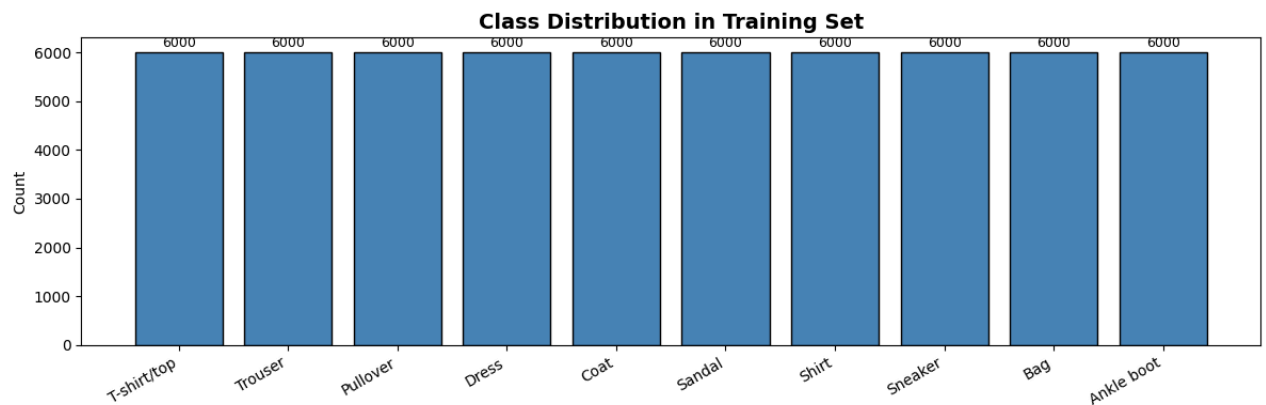
plt.tight_layout()
plt.show()
```

Sample Images — One Per Class



```
# Class distribution
unique, counts = np.unique(y_train, return_counts=True)

plt.figure(figsize=(12, 4))
bars = plt.bar([class_names[i] for i in unique], counts, color='steelblue', edgecolor='black')
plt.title('Class Distribution in Training Set', fontsize=14, fontweight='bold')
plt.ylabel('Count')
plt.xticks(rotation=30, ha='right')
for bar, count in zip(bars, counts):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
             str(count), ha='center', va='bottom', fontsize=9)
plt.tight_layout()
plt.show()
```



3. Data Preprocessing

```
# Normalize pixel values to [0, 1]
X_train = X_train.astype('float32') / 255.0
X_test = X_test.astype('float32') / 255.0

# Reshape to add channel dimension: (samples, 28, 28) -> (samples, 28, 28, 1)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

print(f'X_train shape: {X_train.shape}')
```

```
print(f'X_test shape: {X_test.shape}')
print(f'Pixel value range: [{X_train.min()}, {X_train.max()}]')
```

```
X_train shape: (60000, 28, 28, 1)
X_test shape: (10000, 28, 28, 1)
Pixel value range: [0.0, 1.0]
```

4. Build the CNN Model

```
def build_cnn(input_shape=(28, 28, 1), num_classes=10):
    model = models.Sequential([
        # — Block 1 —————
        layers.Conv2D(32, (3, 3), activation='relu', padding='same',
                      input_shape=input_shape, name='conv1_1'),
        layers.BatchNormalization(),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same', name='conv1_2'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2), name='pool1'),
        layers.Dropout(0.25, name='drop1'),

        # — Block 2 —————
        layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2_1'),
        layers.BatchNormalization(),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same', name='conv2_2'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2), name='pool2'),
        layers.Dropout(0.25, name='drop2'),

        # — Block 3 —————
        layers.Conv2D(128, (3, 3), activation='relu', padding='same', name='conv3_1'),
        layers.BatchNormalization(),
        layers.Dropout(0.25, name='drop3'),

        # — Classifier Head —————
        layers.Flatten(name='flatten'),
        layers.Dense(256, activation='relu', name='fc1'),
        layers.BatchNormalization(),
        layers.Dropout(0.5, name='drop4'),
        layers.Dense(num_classes, activation='softmax', name='output')
    ], name='FashionMNIST_CNN')
    return model

model = build_cnn()
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "FashionMNIST_CNN"

Layer (type)	Output Shape	Param #
conv1_1 (Conv2D)	(None, 28, 28, 32)	320
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
conv1_2 (Conv2D)	(None, 28, 28, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 28, 28, 32)	128
pool1 (MaxPooling2D)	(None, 14, 14, 32)	0
drop1 (Dropout)	(None, 14, 14, 32)	0
conv2_1 (Conv2D)	(None, 14, 14, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 64)	256
conv2_2 (Conv2D)	(None, 14, 14, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 14, 14, 64)	256
pool2 (MaxPooling2D)	(None, 7, 7, 64)	0
drop2 (Dropout)	(None, 7, 7, 64)	0
conv3_1 (Conv2D)	(None, 7, 7, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 7, 7, 128)	512
drop3 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
fc1 (Dense)	(None, 256)	1,605,888
batch_normalization_5 (BatchNormalization)	(None, 256)	1,024
drop4 (Dropout)	(None, 256)	0
output (Dense)	(None, 10)	2,570

Total params: 1,749,610 (6.67 MB)

5. Compile the Model

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print('Model compiled successfully!')
print(f'Total trainable parameters: {model.count_params():,}')
```

Model compiled successfully!
Total trainable parameters: 1,749,610

6. Define Callbacks

```
callbacks = [
    EarlyStopping(
        monitor='val_accuracy',
        patience=5,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3,
        min_lr=1e-6,
        verbose=1
    )
]
```

```
)
]

print('Callbacks defined: EarlyStopping + ReduceLRonPlateau')
```

Callbacks defined: EarlyStopping + ReduceLRonPlateau

7. Data Augmentation

```
# Light augmentation suitable for fashion images
data_augmentation = keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.1),
    layers.RandomTranslation(height_factor=0.05, width_factor=0.05)
], name='augmentation')

# Build augmented training dataset
BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE

train_ds = (
    tf.data.Dataset.from_tensor_slices((X_train, y_train))
    .shuffle(10000)
    .batch(BATCH_SIZE)
    .map(lambda x, y: (data_augmentation(x, training=True), y),
        num_parallel_calls=AUTOTUNE)
    .prefetch(AUTOTUNE)
)

val_ds = (
    tf.data.Dataset.from_tensor_slices((X_test, y_test))
    .batch(BATCH_SIZE)
    .prefetch(AUTOTUNE)
)

print('Augmented training dataset ready!')
```

Augmented training dataset ready!

8. Train the Model

```
EPOCHS = 10

history = model.fit(
    train_ds,
    epochs=EPOCHS,
    validation_data=val_ds,
    callbacks=callbacks,
    verbose=1
)
```

```
Epoch 1/10
938/938 — 373s 389ms/step - accuracy: 0.7594 - loss: 0.6747 - val_accuracy: 0.8164 - val_loss: 0.4947 - 1
Epoch 2/10
938/938 — 375s 382ms/step - accuracy: 0.8338 - loss: 0.4510 - val_accuracy: 0.8631 - val_loss: 0.3846 - 1
Epoch 3/10
938/938 — 387s 387ms/step - accuracy: 0.8544 - loss: 0.3976 - val_accuracy: 0.8706 - val_loss: 0.3484 - 1
Epoch 4/10
938/938 — 356s 379ms/step - accuracy: 0.8648 - loss: 0.3673 - val_accuracy: 0.8859 - val_loss: 0.3096 - 1
Epoch 5/10
938/938 — 382s 379ms/step - accuracy: 0.8728 - loss: 0.3457 - val_accuracy: 0.8903 - val_loss: 0.3070 - 1
Epoch 6/10
938/938 — 380s 377ms/step - accuracy: 0.8810 - loss: 0.3299 - val_accuracy: 0.8975 - val_loss: 0.2871 - 1
Epoch 7/10
938/938 — 354s 378ms/step - accuracy: 0.8856 - loss: 0.3153 - val_accuracy: 0.9042 - val_loss: 0.2635 - 1
Epoch 8/10
938/938 — 356s 379ms/step - accuracy: 0.8860 - loss: 0.3098 - val_accuracy: 0.9073 - val_loss: 0.2568 - 1
Epoch 9/10
938/938 — 354s 377ms/step - accuracy: 0.8919 - loss: 0.2963 - val_accuracy: 0.9072 - val_loss: 0.2602 - 1
Epoch 10/10
938/938 — 356s 380ms/step - accuracy: 0.8927 - loss: 0.2920 - val_accuracy: 0.8981 - val_loss: 0.2776 - 1
Restoring model weights from the end of the best epoch: 8.
```

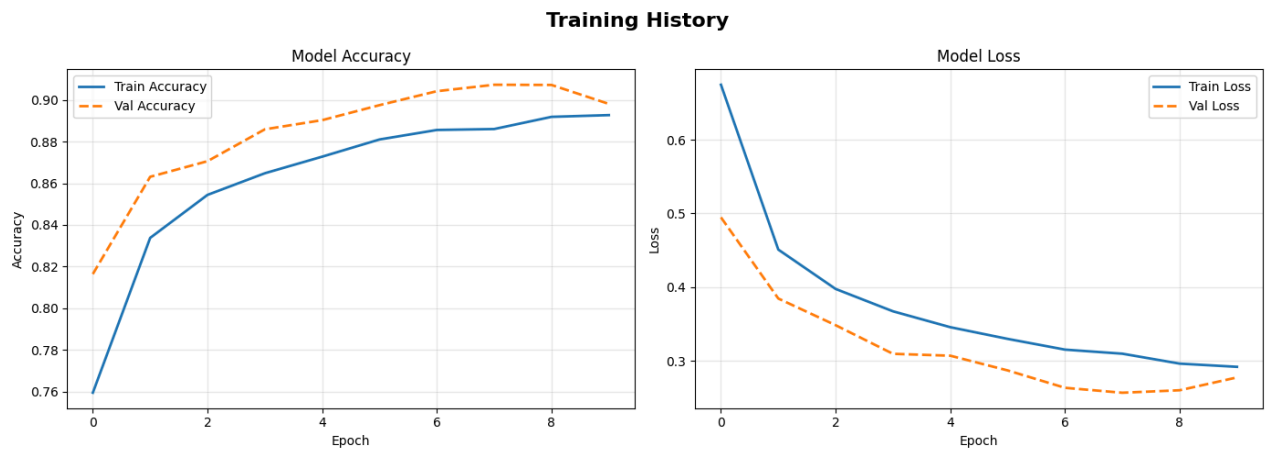
9. Plot Training History

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Training History', fontsize=16, fontweight='bold')

# Accuracy
axes[0].plot(history.history['accuracy'], label='Train Accuracy', linewidth=2)
axes[0].plot(history.history['val_accuracy'], label='Val Accuracy', linewidth=2, linestyle='--')
axes[0].set_title('Model Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Loss
axes[1].plot(history.history['loss'], label='Train Loss', linewidth=2)
axes[1].plot(history.history['val_loss'], label='Val Loss', linewidth=2, linestyle='--')
axes[1].set_title('Model Loss')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



10. Evaluate the Model

```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Loss : {test_loss:.4f}')
print(f'Test Accuracy: {test_accuracy:.4f} ({test_accuracy*100:.2f}%)')
```

```
Test Loss : 0.2568
Test Accuracy: 0.9073 (90.73%)
```

11. Classification Report

```
y_pred_probs = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_pred_probs, axis=1)

print('Classification Report')
print('=' * 60)
print(classification_report(y_test, y_pred, target_names=class_names))
```

```
Classification Report
=====
              precision    recall  f1-score   support

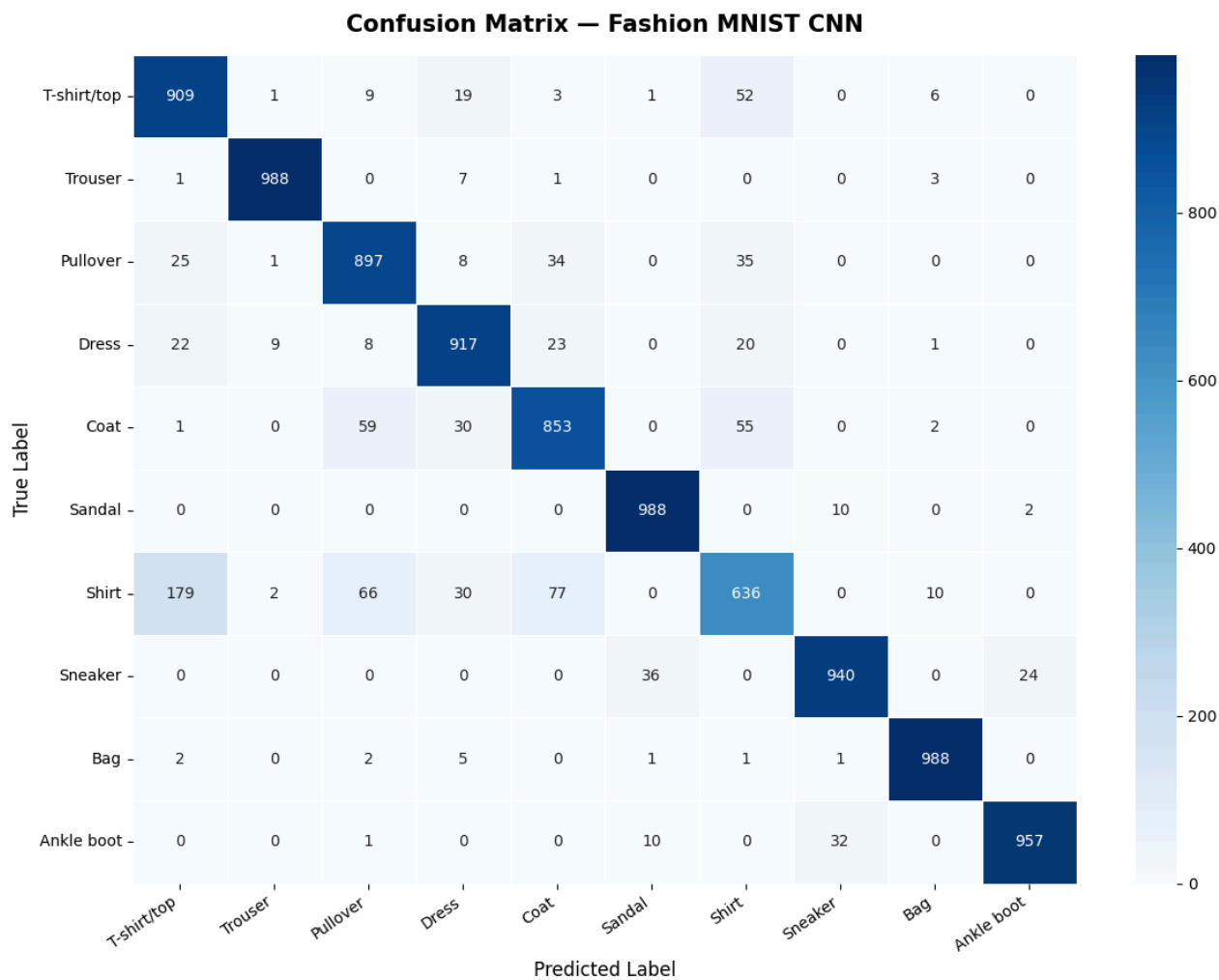
T-shirt/top      0.80      0.91      0.85       1000
Trouser          0.99      0.99      0.99       1000
Pullover         0.86      0.90      0.88       1000
Dress            0.90      0.92      0.91       1000
Coat             0.86      0.85      0.86       1000
Sandal           0.95      0.99      0.97       1000
Shirt            0.80      0.64      0.71       1000
```

Sneaker	0.96	0.94	0.95	1000
Bag	0.98	0.99	0.98	1000
Ankle boot	0.97	0.96	0.97	1000
accuracy			0.91	10000
macro avg	0.91	0.91	0.91	10000
weighted avg	0.91	0.91	0.91	10000

12. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(12, 9))
sns.heatmap(
    cm,
    annot=True, fmt='d',
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names,
    linewidths=0.5
)
plt.title('Confusion Matrix - Fashion MNIST CNN', fontsize=15, fontweight='bold', pad=15)
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('True Label', fontsize=12)
plt.xticks(rotation=35, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



```

n = 20
indices = np.random.choice(len(X_test), n, replace=False)

fig, axes = plt.subplots(4, 5, figsize=(15, 12))
fig.suptitle('Sample Predictions (Green = Correct, Red = Wrong)', fontsize=14, fontweight='bold')

for i, ax in enumerate(axes.flat):
    idx = indices[i]
    true_label = class_names[y_test[idx]]
    pred_label = class_names[y_pred[idx]]
    confidence = y_pred_probs[idx][y_pred[idx]] * 100
    correct = y_test[idx] == y_pred[idx]

    ax.imshow(X_test[idx].reshape(28, 28), cmap='gray')
    color = 'green' if correct else 'red'
    ax.set_title(
        f'True : {true_label}\nPred : {pred_label}\nConf : {confidence:.1f}%',
        fontsize=8, color=color
    )
    ax.axis('off')

plt.tight_layout()
plt.show()

```

Sample Predictions (Green = Correct, Red = Wrong)

